

Data Tooling - Coding Assignment

In this assignment you will fetch data from the SEC's EDGAR API, a free data source that powers parts of our production systems, and build an API and UI on top of it. You may find the documentation on how the API works [here](#) and [here](#).

Practicalities

- **Time budget:** the assignment is scoped to be completed in about 4 hours, including project setup. If you find yourself well past that, stop, submit what you have, and include a short [NOTES.md](#) describing what is missing and how you would finish it.
- The submission must be written in **TypeScript**, with a **React** frontend. Backend framework and libraries are your choice. We build with Bun and Elysia but you are welcome to use any TypeScript library.
- You may use any tools you like, including AI tools.

The task

Your API should work for any company that files with the SEC. A few to try it with: Apple, Spotify, JPMorgan Chase.

Backend: build a small API that:

1. Fetches a company's filing history from the EDGAR submissions API (data.sec.gov/submissions/CIK#####.json) given its ticker. You will need to resolve tickers to CIK numbers yourself (hint: [company_tickers.json](#)).
2. Normalizes the response into a list of filing objects. Note that EDGAR returns filings in a columnar format, not as an array of objects.
3. Exposes the filings through **GET /companies/:ticker/filings**: a paginated list that can be filtered by form type (e.g. 10-K, 10-Q, 8-K), where each filing links to its original document on [sec.gov](#).

4. Exposes a summary through **GET /filings/summary**: given a set of companies, it returns each company's number of filings per form type over the last 12 months and the date of its latest 10-K.

Frontend: build a simple UI on top of your API that:

1. Lists filings with controls to switch company, filter by form type, and sort by filing date.
2. Displays the summary view in whatever form you find most useful.

Extensions: if you finish with time to spare, extend the solution in whatever direction you think adds the most value.

Submission

The submission should contain:

- Source code, with instructions on how to run it.
- The prompt log, if any AI tool was used.
- When done commit your code to a repository in Github and send your submission our way for us to take a look!

Afterwards we will book a ~60 minute review session where we walk through your solution together. Come prepared to run your solution and explain the decisions behind it.

Good luck!